AIAMM Whitepaper and Implementation Guide

Oct 29, 2026 · @ff

Executive summary

Reinforcement-learning agents can manage a Uniswap v3 liquidity position autonomously, but on real ETH/USDC data they did not beat simply leaving liquidity in place. They added value only when volatility was high, and that has been shown in simulation only.

AIAMM (MCAST Research Fellowship Scheme 2025, with Web3 Media) trained three agents (Double DQN, PPO and SAC) to decide where a liquidity provider's capital should sit. Each agent was benchmarked against a passive full-range position on 38,495 hourly ETH/USDC prices from May 2021 to March 2026.

DDQN was the strongest of the three, but it trailed passive provision by $6.29 per 500-hour episode on $2,000 of capital (95% CI −6.50 to −6.09). It came out ahead in only 12.4% of episodes. When simulated volatility rose to $\sigma = 0.030$ per step and above, the same agent outperformed passive provision by $53.62 to $240.87 per episode.

On 28 October 2026 the trained agent ran end-to-end on the Ethereum Sepolia testnet against the official Uniswap v3 contracts. This guide explains the method and results, how to run the published code, and when active management is worth switching on.

The problem: where to place liquidity

In a concentrated-liquidity AMM such as Uniswap v3, a liquidity provider chooses a price range. Capital earns swap fees only while the price stays inside that range.

A narrow range earns more fees per dollar, but the price leaves it more often. Once out of range, the position earns nothing and is held entirely in the weaker asset. A wide or full range keeps earning, but less intensively.

Every move of the range costs gas and locks in impermanent loss: the value lost against simply holding the two tokens when their prices diverge. The provider's question is therefore a sequential decision under uncertainty: hold, add or remove liquidity, or shift the range. That is the kind of problem reinforcement learning is built for.

AIAMM asked whether a trained agent can make those decisions better than a passive full-range position once costs are counted.

Approach

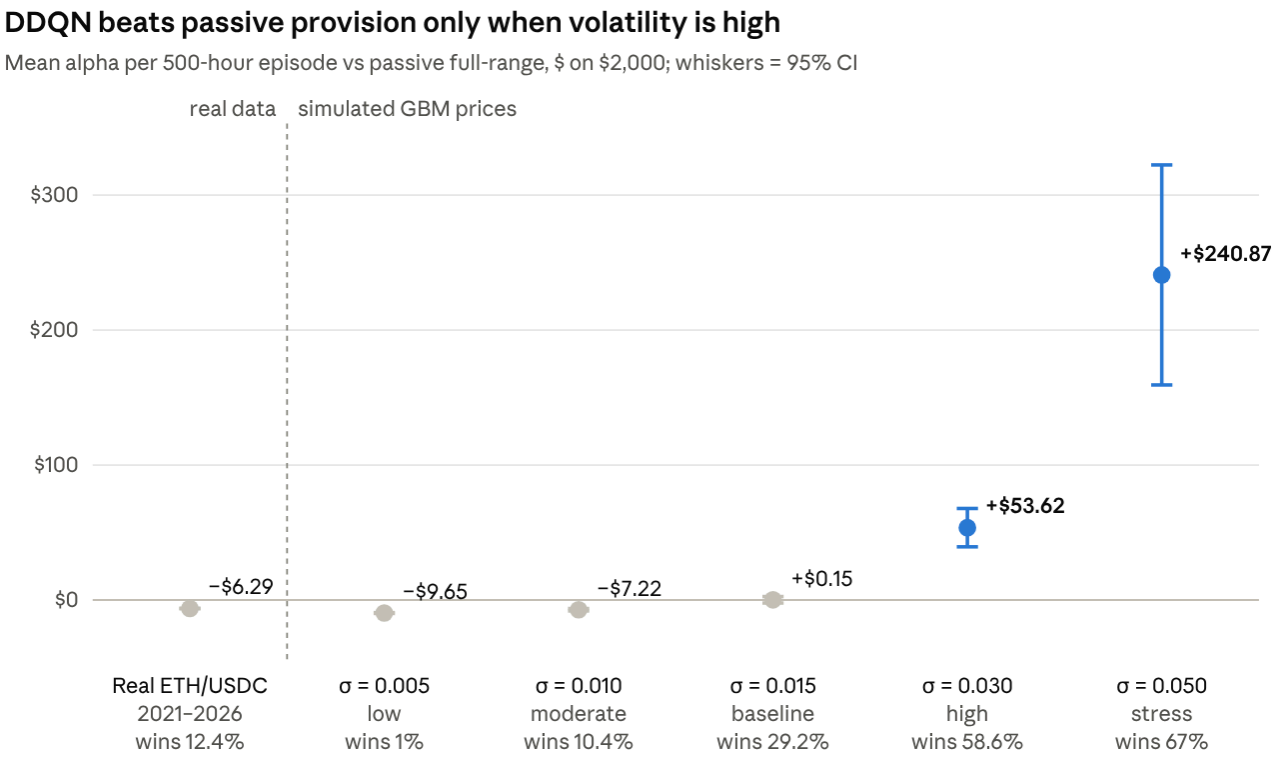
The agents learn inside a Python simulation of a Uniswap v3-style pool (`AMMEnvironment` in `13_AIAMM_V13.py`). The simulation charges swap fees, lets arbitrageurs pull the pool to the market price, and charges a fixed gas cost per action.

Component	Design
Market data	38,495 hourly ETH/USDC candles, Binance public API, 3 May 2021 – 6 March 2026; synthetic GBM paths for the volatility sweep
State (15 inputs)	Price and AMM returns, rolling volatility, range position, liquidity ratio, fees earned, alpha signal, price dislocation, momentum, a price-derived sentiment indicator, episode progress, cooldown, high–low range, open–close return, volume ratio
Actions (5)	Hold; increase or decrease liquidity by 10% of capital; shift the ±40% range up or down by 3%
Reward	Weighted sum of alpha versus passive (0.571), fee income (0.286), impermanent-loss penalty (0.095) and transaction-cost penalty (0.048), clipped to [−3, +3]
Costs	$0.15 per liquidity change, $0.30 per range shift
Agents	Double DQN (128/64 hidden units, hard target sync every 500 updates), PPO and SAC, learning rates 1e-3 (DDQN) and 3e-4
Training	1,000 episodes per seed with a curriculum of 100, 300 and then 500 hourly steps
Evaluation	5 seeds; mean alpha with 95% CIs; win rate; Sharpe and Sortino of alpha; one-sample t-tests with Bonferroni correction; Earliest Effective Cutoff bisection; a five-level GBM volatility sweep

Alpha is the agent's terminal portfolio value minus that of a passive full-range position on the same price path. A positive alpha means the agent beat doing nothing.

Results

On real ETH/USDC data none of the three agents beat passive provision after costs. DDQN, the best of the three, became profitable only in simulated markets with high volatility.



AIAMM V13 final run, 22 May 2026: real data 5 seeds × 1,000 episodes; GBM sweep 5 seeds × 100 evaluation episodes per σ

PPO (−$7.09) and SAC (−$7.11) trailed DDQN on real data, and all three shortfalls were significant after Bonferroni correction ($p_{adj} < 0.001$). The Earliest Effective Cutoff was never reached: at no horizon up to 500 hours did DDQN win half its episodes. The gap between real data and the matched-volatility simulation comes from trends. Real ETH/USDC prices move directionally, which favours a position that stays fully in range.

An October 2026 re-run of the unchanged code added the measures the original proposal named. Against passive provision, DDQN on real data:

- earned less in fees ($6.56 vs $9.70 per episode);
- kept 68.1% of its capital in range;
- had slightly less impermanent loss (0.11 percentage points);
- had a 95% Value-at-Risk of alpha of −$15.54;
- lost money in bull, bear and sideways phases alike, least in the most volatile third of episodes (−$3.74).

On-chain demonstration (Ethereum Sepolia, 28 October 2026)

The trained DDQN agent ran a full 48-hour episode on a public blockchain without human intervention. Every decision became real Uniswap v3 transactions that anyone can verify on sepolia.etherscan.io.

Two zero-value test tokens and a 0.30% pool were created through the official Uniswap v3 Factory, and three positions were minted:

- background liquidity;
- a passive full-range benchmark;
- the agent's ±40% position.

Each step swapped the pool to the next real hourly ETH/USDC price from 26–28 February 2026. The agent then read the on-chain state and acted. It held 44 times and removed liquidity 4 times. In this low-volatility window ($\sigma \approx 0.0045$ per hour) it finished at $1,932.43 against $1,934.07 for the passive position (alpha −$1.64, valued from chain state), as the simulation predicts for calm markets.

Three lessons matter for anyone deploying such an agent:

1. **The action space maps cleanly to the chain.** Each action is one or two `NonfungiblePositionManager` calls (`decreaseLiquidity` + `collect` , `increaseLiquidity` , or burn-and- `mint` for a range shift), each with an auditable log.
2. **Public RPC endpoints fail after broadcasting.** Twice a `collect` succeeded on-chain but its receipt could not be read. The script retried, and the retry collected nothing. The bookkeeping then under-recorded the agent by $329.41 (alpha −$331.05 instead of the true −$1.64). Never re-send a transaction that has been broadcast. Poll its hash, and reconcile balances against on-chain events. The published kit now does both (`tn_common.Chain.send` , `04_reconcile.py`).
3. **A testnet proves plumbing, not profit.** Test pools have no organic order flow, so fees come only from the replay swaps. One episode is not a statistical test.

Implementation guide

Everything needed to reproduce the study and the on-chain run is in the public repository github.com/francofarrugia/AIAMM under the MIT licence.

Reproduce the simulation results

1. Install Python 3.9+ and `pip install numpy scipy matplotlib` .
2. In `simulation/` , run `python 13_AIAMM_V13.py` . It trains DDQN, PPO and SAC on five seeds and prints Tables II–V. Expect several hours on a laptop.
3. To rebuild the dataset from Binance, run `python fetch_uniswap_data.py` .

Repeat the Sepolia run (about one hour, no real money)

1. In `testnet/`, run `pip install --only-binary=:all: -r requirements.txt`.
2. Run `python 00_create_wallet.py`. It writes `wallet.json` locally; never commit or share it.
3. Fund the printed address with free Sepolia ETH from a faucet. 0.05 test ETH is plenty.
4. Run `python 01_deploy.py` to create the test tokens, the pool and the three positions (10 transactions).
5. Run `python 02_run_agent.py --steps 48`. It checkpoints after every step; rerun the same command to resume.
6. Run `python 03_report.py`, then `python 04_reconcile.py`. These write the evidence report, CSVs and chart, checked against the chain.

Adapt to another pool or pair

- Replace the price file with hourly candles for your pair (`open`, `high`, `low`, `close`, `volume`, `open_times`) and retrain with `model/train_export.py`.
- Set `GAS_MODEL` in `02_run_agent.py` to the real cost of each action on your chain. On a mainnet this decides whether active management can pay at all.
- For a different fee tier or range width, change `FEE` / `TICK_SPACING` in `tn_common.py` and the ±40% band in the deploy and run scripts. Retrain, because the agent learned that band.
- Keep the passive full-range benchmark in every run. Alpha against it is the only number that says whether the agent earned its costs.

Operating guidance for liquidity providers

Use a passive full-range position by default. Consider an RL range manager only in high-volatility regimes, and only after testing it on your own pair and costs.

Market condition (hourly σ)	Evidence from AIAMM	Guidance
Calm, below about 0.010	DDQN loses $7–10 per episode in simulation; 1–10% win rate	Stay passive
Moderate, about 0.015	Break-even in simulation (+$0.15, not significant)	Stay passive; costs decide
Real ETH/USDC, 2021–2026	Every agent below passive; best −$6.29, 12.4% win rate	Stay passive

Market condition (hourly σ)	Evidence from AIAMM	Guidance
High, 0.030 and above	DDQN +$53.62 to +$240.87 per episode, 58.6–67% win rate (simulation only)	Candidate for active management after live validation

Three checks come before any live deployment:

1. Measure the realised volatility of your pair over recent windows. If it sits below the break-even level, an agent will mostly pay gas for nothing.
2. Price gas realistically. AIAMM assumed $0.15–$0.30 per action; on Ethereum mainnet one rebalance can cost far more, which moves break-even upward.
3. Run the agent next to a passive benchmark on a testnet, then with small capital, and reconcile every balance against the chain.

Limitations and future work

The positive results are simulation results. Four gaps stand between them and a production system.

- **No NLP sentiment.** The agents' sentiment input is computed from recent price momentum. Removing it changed nothing ($p = 0.88$). Whether news, social or on-chain text adds information is untested.
- **Exchange prices, not pool data.** Swap volume, fee income and arbitrage are simulated, not observed from Uniswap pools.
- **Not measured:** slippage, capital turnover and arbitrage profitability. These need tick-level pool liquidity and transaction ordering.
- **No live deployment.** Mainnet gas, MEV and real order flow remain untested.

The next study would train on on-chain pool data, add a validated text-sentiment feed, and price gas per chain. It would then run a small-capital live pilot against a passive benchmark, restricted to high-volatility regimes.

Sources

- Farrugia, F. & Deguara, C., *Reinforcement Learning Agents for Adaptive Liquidity Provisioning in Concentrated Decentralised Exchanges: A Comparative Architecture Study*, MCAST Journal of Applied Research and Practice (accepted 27 October 2026).
- AIAMM Periodic Research Project Report IV, revised October 2026 (Sections 4–6, Appendices I, M, N).

- Code, data and testnet evidence: github.com/francofarrugia/AIAMM.